

LECTURE 18

Normalization: Purpose and Concepts

Why some relation designs quietly corrupt themselves — and how to know

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

From Redundancy to a Fixable Problem

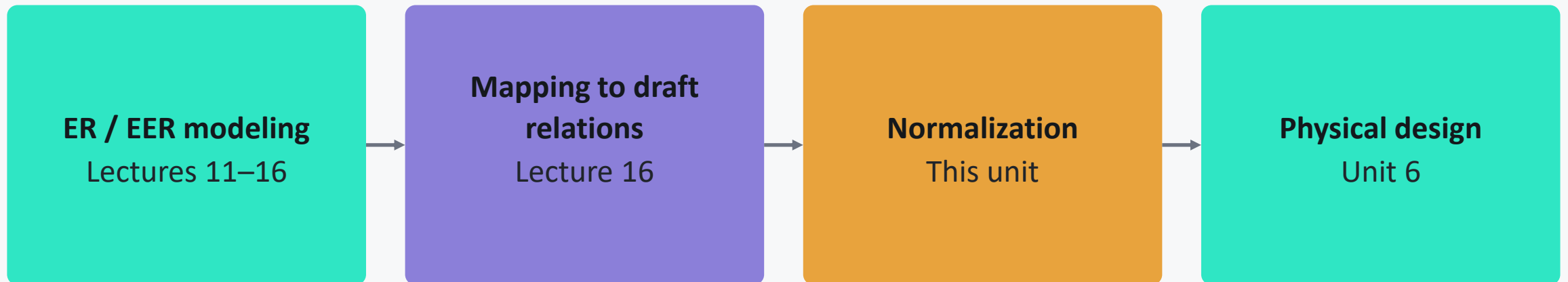
- 1 What normalization is for — not “make it neat,” but eliminate provable problems
- 2 Data redundancy: a concrete, worked example of a poorly designed relation
- 3 The three update anomalies — insertion, deletion, modification
- 4 Functional dependencies: the notation $X \rightarrow Y$ that makes “good design” checkable
- 5 Decomposition, and its two required properties: lossless-join & dependency-preservation
- 6 The overall shape of the normalization process — 1NF through BCNF/4NF

What Normalization Is For

Normalization is a formal technique for analyzing a relation based on its functional dependencies, and systematically decomposing it into smaller relations that are provably free of certain kinds of redundancy — and the anomalies that redundancy causes.

NOTE

Normalization does not tell you WHAT entities or relationships your database needs — that's ER/EER modeling's job. It tells you whether the relations you already derived are well-structured, and how to repair them if they aren't.



Data Redundancy: A Worked Example

A rushed designer proposes one wide relation to record orders and their products:

Order (orderNo, orderDate, custNo, custName, custAddress, productNo, productName, unitPrice, qty)

orderNo	custNo	custName	custAddress	productNo	productName	qty
O100	C001	Ann Beech	12 Elm St, Lahore	P01	Widget	3
O100	C001	Ann Beech	12 Elm St, Lahore	P02	Gadget	1
O101	C002	Tom Kelly	5 Oak Ave, Karachi	P01	Widget	2
O102	C001	Ann Beech	12 Elm St, Lahore	P03	Sprocket	10

Ann Beech's name/address repeats in 3 rows; **Widget**'s name/price repeats in 2 rows — the same fact, stored redundantly, because this one relation describes three different kinds of things (orders, customers, products) at once.

Spotting the Redundancy, Row by Row

Same four rows as the previous slide. Before the anomalies, trace each repeated value by hand:

Rows 1, 2, 4 share custNo = C001. Rows 1, 3 share productNo = P01.

orderNo	custNo	custName	custAddress	productNo	productName	qty
O100	C001	Ann Beech	12 Elm St, Lahore	P01	Widget	3
O100	C001	Ann Beech	12 Elm St, Lahore	P02	Gadget	1
O101	C002	Tom Kelly	5 Oak Ave, Karachi	P01	Widget	2
O102	C001	Ann Beech	12 Elm St, Lahore	P03	Sprocket	10

custName / custAddress repeat in rows 1, 2, and 4 — three copies of one customer fact (“Ann Beech”, 12 Elm St, Lahore). productName repeats in rows 1 and 3 — two copies of one product fact (“Widget”). Nothing in this table keeps the copies in sync.

Insertion Anomaly — New Product

Product P04 (“Cog”) hasn't been ordered yet. **There is no way to insert this fact** into the Order relation — every row requires an orderNo, and there is no order to attach it to.

A product should be able to exist in the catalog before its first sale. Attempted row: (orderNo = —, productNo = P04, productName = Cog, qty = —) → REJECTED: orderNo cannot be NULL (entity integrity).

Why this matters:

The catalog and the sales history are two different kinds of facts, but this one relation fuses them — you cannot record one without faking the other.

Insertion Anomaly — New Customer

A brand-new customer, C003 (“Zara Malik”), has registered but hasn’t placed an order yet. **There is no way to insert this fact** into the Order relation either — custName and custAddress only ever exist attached to an orderNo, and there is no order yet.

Attempted row: (orderNo = —, custNo = C003, custName = Zara Malik, custAddress = 9 Mall Rd, Lahore, productNo = —, qty = —) → REJECTED: orderNo cannot be NULL (entity integrity).

Why this matters:

A real system needs to record leads and sign-ups before their first purchase — this schema structurally cannot.

Deletion Anomaly

Order O101 — Tom Kelly’s ONLY order — is cancelled, so its row is deleted:

Deleting one row should remove one fact (“an order was placed”). Watch what else goes.

orderNo	custNo	custName	custAddress	productNo	productName	qty
O100	C001	Ann Beech	12 Elm St, Lahore	P01	Widget	3
O100	C001	Ann Beech	12 Elm St, Lahore	P02	Gadget	1
O101	C002	Tom Kelly	5 Oak Ave, Karachi	P01	Widget	2
O102	C001	Ann Beech	12 Elm St, Lahore	P03	Sprocket	10

Tom Kelly’s row is the ONLY place his name and address exist anywhere in this database. Deleting it doesn’t just cancel one order — it erases the customer record entirely, even though “Tom Kelly exists” and “O101 was ordered” are two unrelated facts that should never live or die together.

Modification Anomaly

Modification anomaly — Widget’s (P01) catalog price changes from 9.99 to 11.49. That single fact is stored in TWO rows of this table — one per order line that included Widget.

Before vs. after: updating only O100’s row — easy to do by accident — leaves O101’s copy stale:

orderNo	productNo	productName	unitPrice
O100	P01	Widget	11.49 (updated)
O101	P01	Widget	9.99 (STALE — should be 11.49)

WARNING

The same product now has two different prices depending on which row you query — easy to trigger by accident through application code that updates only “the row the user is looking at.”

Root Cause: Why All Three Happen

Insertion, deletion, and modification are not three unrelated bugs. **They share exactly one structural cause** — and once you name it, none of the three stays surprising.

Every one of the last four slides failed because ONE fact — a customer's address, a product's price, an order's existence — was stored in MORE THAN ONE row, with nothing in the table's structure keeping the copies in sync.

All three anomalies trace to one root cause:

this one relation forces facts about three different kinds of things — orders, customers, and products — to live and die together, when in reality they don't.

Functional Dependencies

NOTE

Given relation R and attribute sets $X, Y \subseteq R$, Y is functionally dependent on X — written $X \rightarrow Y$, read “ X functionally determines Y ” — if every legal value of X is associated with exactly one value of Y . If two tuples agree on X , they must agree on Y too. X is the determinant.

Reading the FDs actually present in the **Order** relation:

```
orderNo → orderDate, custNo  
custNo → custName, custAddress  
productNo → productName, unitPrice  
{ orderNo, productNo } → qty
```

An FD is a statement about the SCHEMA, not about today's data — it must hold for every possible legal instance, not just the sample rows shown.

Verifying the FDs Against Real Rows

CHECK #1

custNo $\rightarrow$ custName, custAddress: rows with custNo = C001 (rows 1, 2, 4) all show “Ann Beech” and “12 Elm St, Lahore”. This holds by business rule, not because the sample rows happen to agree.

CHECK #2 — does orderNo ALONE determine qty?

orderNo = O100 is on TWO rows (qty 3 with P01, qty 1 with P02): orderNo alone does NOT fix qty.
productNo = P01 is on TWO rows (qty 3 with O100, qty 2 with O101): productNo alone does NOT fix qty either.
Only the PAIR {orderNo, productNo} picks out exactly one row, so only the pair determines qty.
That is exactly what { orderNo, productNo } $\rightarrow$ qty means, and why BOTH attributes are on the left.

This distinction — qty needs the WHOLE key; customer/product facts need only PART of it — is precisely what Lecture 19 formalizes as 2NF.

Functional Dependency and Normalization

Every normal form from this point on is defined purely in terms of which FDs hold and how they relate to the relation's keys:

Normal Form	Asks...
1NF	Is every attribute value atomic? (independent of FDs)
2NF	Does every non-key attribute depend on the WHOLE key, or only part (partial dependency)?
3NF	Does every non-key attribute depend DIRECTLY on the key, or only through another non-key attribute (transitive)?
BCNF	Is EVERY determinant a superkey, with no exceptions?

Without functional dependencies, “well-designed” stays a matter of taste. With them, it becomes something you can prove.

Decomposition of Relations

Decomposition replaces one relation with several that together keep the same information. Preview:

Customer (custNo, custName, custAddress)

custNo	custName	custAddress
C001	Ann Beech	12 Elm St, Lahore
C002	Tom Kelly	5 Oak Ave, Karachi

Product (productNo, productName, unitPrice)

productNo	productName	unitPrice
P01	Widget	9.99
P02	Gadget	24.99
P03	Sprocket	4.50

Order (orderNo, orderDate, custNo)

orderNo	orderDate	custNo
O100	2026-08-01	C001
O101	2026-08-02	C002
O102	2026-08-03	C001

OrderLine (orderNo, productNo, qty)

orderNo	productNo	qty
O100	P01	3
O100	P02	1
O101	P01	2
O102	P03	10

Which FD Drove Which Split

On the previous slide, Order was decomposed into four relations. Each FD became the primary key of exactly one of them:

Functional Dependency	Enforced By
orderNo → orderDate, custNo	Order (orderNo, orderDate, custNo)
custNo → custName, custAddress	Customer (custNo, custName, custAddress)
productNo → productName, unitPrice	Product (productNo, productName, unitPrice)
{orderNo, productNo} → qty	OrderLine (orderNo, productNo, qty)

Every attribute of the original Order relation lands in exactly one decomposed relation, and every FD is enforced there by a simple key constraint — no application code required.

Lossless-Join Property

A decomposition of R into $R_1, R_2, \dots, R_n$ has the **lossless-join property** if the natural join $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$ reconstructs exactly R — no rows lost, and no extra spurious rows gained.

Customer $\bowtie$ Order $\bowtie$ OrderLine $\bowtie$ Product reproduces the original Order table exactly

WARNING

The classic way to lose losslessness: split on the wrong attribute. If Order were split into (orderNo, custNo, productNo) and (productNo, productName, unitPrice, qty) — dividing attributes without regard to which FDs justify the split — rejoining on productNo alone would produce every combination of order and quantity for a product, most of which never happened. A correct decomposition is always guided by functional dependencies, never an arbitrary attribute split.

Lossless-Join: A Wrong Split

A split is lossless only if rejoining the pieces recreates exactly the original rows. **This split ignores the FDs** and produces ghost rows.

Wrong split: R1(orderNo, custNo, productNo) and R2(productNo, qty), joined on productNo alone.

WHAT GOES WRONG

P01 has 2 rows in R1 and 2 rows in R2, so the join makes $2 \times 2 = 4$ P01 combinations. Only 2 are real; the other 2 are spurious.

orderNo	custNo	productNo	qty	status
O100	C001	P01	3	real (true row)
O100	C001	P01	2	SPURIOUS: O100 never bought P01 at qty 2
O101	C002	P01	3	SPURIOUS: O101 never bought P01 at qty 3
O101	C002	P01	2	real (true row)
O100	C001	P02	1	real (true row)
O102	C001	P03	10	real (true row)

Dependency-Preservation Property

A decomposition has the **dependency-preservation property** if every FD from the original relation's FD set can still be checked directly on the decomposed relations — without joining them back together first.

In the Order decomposition: $\text{custNo} \rightarrow \text{custName}$ is checkable directly on Customer alone (declare custNo its PK, the DBMS enforces it automatically), and $\{\text{orderNo}, \text{productNo}\} \rightarrow \text{qty}$ is checkable directly on OrderLine. Every original FD survives onto exactly one decomposed relation — lossless AND dependency-preserving.

TIP

This matters practically: a dependency that can only be verified after a join is one the DBMS cannot enforce cheaply with a simple key constraint on one table. Lecture 20 shows this property isn't always achievable simultaneously with the strictest normal form, BCNF — a genuine trade-off.

The Normalization Process, Staged

Unnormalized (UNF)

May contain repeating groups / non-atomic values



1NF

Atomic values, no repeating groups (Lecture 19)



2NF

No partial dependency on a composite key (Lecture 19)



3NF

No transitive dependency (Lecture 19)



BCNF

Every determinant is a superkey (Lecture 20–21)



4NF

No non-trivial multi-valued dependency (Lecture 22)

KEY TAKEAWAYS

Key Takeaways

- 1 Normalization is a formal, FD-driven technique for validating and refining relations produced during ER/EER design — a check, not a replacement
- 2 Redundancy is the root cause of the three update anomalies: insertion, deletion, modification
- 3 A functional dependency $X \rightarrow Y$ formalizes “X determines Y” precisely enough to check mechanically — every later normal form is defined in terms of FDs and keys
- 4 Decomposition must satisfy the lossless-join property and, ideally, dependency-preservation
- 5 The process is fixed and increasingly strict: $UNF \rightarrow 1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF \rightarrow 4NF$, each removing one specific category of dependency problem